

Report: rstac CQL2 extension development

Gilberto Camara, Rolf Simoes, and Felipe Carvalho

2023-01-23

Introduction

Earth observation images provide a reliable and consistent way to monitor Earth's surface. In the last decades, the number of orbital instruments for Earth observation has increased (Finer et al. 2018). Cloud computing has been adopted by industry and academy to store, access, process, and distribute big Earth observation data (Wagemann et al. 2021). In this context, the Spatio-temporal Assets Catalog (STAC) [<https://stacspec.org>] protocol has become a standard to describe and search EO data cloud service providers.

The STAC protocol complies with the OGC API Features standard (OAFeat) (OGC 2019). STAC enables users to search and access satellite images. The STAC specification allows the development of new extensions to support more specific and complex functionality, including the Common Query Language 2 (CQL2) extension (OGC 2022). CQL2 is part of OAFeat standard; it provides a complex way to search and retrieve spatial-temporal assets. CQL2 is a powerful query language that enables users to create complex filtering criteria in a expressive way. Using CQL2 in a STAC service, users can search for spatial assets using a variety of metadata information such as cloud cover and spatial relations.

This report describes the development of an updated version of the rstac R package (Simoes et al. 2021) that provides full support for the CQL2 specification as part of queries that use the STAC protocol. The rstac package can be accessed at <https://github.com/brazil-data-cube/rstac> and is also available on the Comprehensive Archive Network (CRAN).

Project milestones

This section describes the implementation of the authors proposal that was approved by the Radiant Earth Foundation and STAC Project Steering Committee for the advance of STAC ecosystem programme.

Milestone 1 - Proof of concept version with examples that validate the idea of using the package

Status: concluded in July 2022.

We used the abstract syntax tree (AST) of R language to develop the CQL2 extension in `rstac`. In computer science, AST is a way to represent an language expression in a formal way using data structures. AST representation informs the precedence on which and how a identifier will be evaluated. AST can be used to translate code from a language to other (Wickham, Hadley 2019). The advantage of this approach is that users can focus on filtering criteria putting less effort in checking the syntax consistency. Using R environments and lexical scoping, AST gave us a powerful way for translating R code into CQL2 query. This approach showed to be consistent with CQL2 standard. We provide some examples of code translation in appendix A.

Milestone 2 - Implementing integration testing with STAC services supporting CQL2

Status: concluded in September 2022.

The CQL2 standard includes a set of conformance tests (OGC 2019). We have validated our implementation using all conformance tests that provide a test dataset. The test dataset used in `rstac` conformance tests is provided by “Test Dataset for the CQL2 Test Suite” [<https://cql2test.ldproxy.net/ne110m4cql2>], derived from Natural Earth [<https://www.naturalearthdata.com>]. Besides this test suite, we also have implemented tests using Microsoft Planetary Computer STAC service [<https://planetarycomputer.microsoft.com/api/stac/v1>]. The list of conformance tests implemented in `rstac` is listed in appendix B.

Milestone 3 - Pre-release version with documentation and testing

Status: concluded in November 2022.

We pushed a pre-released version of `rstac` into its Github repository. This version passed in all conformance tests referenced in previous topic. CQL2 filtering functionality is implemented in `ext_filter()` function. We have write a function documentation available in `?ext_filter()` in R console. Besides function description and parameters details, the documentation provides examples on how to use CQL2 filtering function using `rstac` package in actual STAC services.

Milestone 4 - CRAN submission and tutorial creation for users and developers.

Status: concluded in January 2023.

We have submitted `rstac` 0.9.2 version to CRAN; R users can now install the package directly from CRAN. Package documentation has been updated with two new tutorials. The first tutorial shows how to create CQL2 expressions with `rstac`. There is also a tutorial on how to access Planetary Computer data using `rstac` and CQL2. We will submit this tutorial to Planetary Computer repository [<https://github.com/microsoft/PlanetaryComputerExamples>].

Concluding remarks

This report describes the results of the CQL2 extension development in `rstac` package. The project enumerated four milestones that were concluded as described above. Current version of the `rstac` package can be accessed at [<https://github.com/brazil-data-cube/rstac>] under MIT

license. The rstac package is open to the community and follow the "Contributor Covenant - version 1.0.0" code of conduct. We updated package's documentation and have prepared new tutorials on how to use rstac and its new CQL2 functionality.

Appendix A - CQL2 examples

Introduction

CQL2 is an OGC standard that enables complex filter expressions on OAFeat3 or STAC web services. CQL2 standard states that expressions can be represented in JSON or TEXT formats. Our implementation intends to convert native R expressions into CQL2 valid expressions without needing cumbersome nested lists or dictionaries. Also, we can make CQL2 filter requisition in JSON or TEXT formats with the same filter representation.

Translating R expressions to CQL2 syntax

To explain the difference between the TEXT and JSON CQL2 representation, lets start with a simple example. In the following code, we have a valid CQL2 expression (in TEXT format) which refers to two properties, vehicle_height and bridge_clearance.

```
vehicle_height > (bridge_clearance - 1))
```

This filter expression can be passed in the HTTP GET verb implemented by service to retrieve only those features that satisfy the condition. The same expression can be represented in JSON format, which is more suitable for HTTP POST requests:

```
{
  "op": ">",
  "args": [
    {"property": "vehicle_height"},
    {
      "op": "-",
      "args": [
        {"property": "bridge_clearance"},
        1
      ]
    }
  ]
}
```

Note how properties vehicle_height and bridge_clearance are represented in this format. They are elements of an object containing a property member. Also, they go as arguments of operators (in this case, > and - operators).

In the R language, the JSON above could be represented by nested lists, which would be somewhat cumbersome to write. To produce valid CQL2 filter expressions, we use the R abstract syntax tree (AST) from R expressions that can be converted to TEXT or JSON formats. Let us see the same previous example written in R CQL2:

```
library(rstac)
```

```
cql2_text(vehicle_height > (bridge_clearance - 1)) # TEXT format  
#> vehicle_height > bridge_clearance - 1
```

```
cql2_json(vehicle_height > (bridge_clearance - 1)) # JSON format  
#> {"op": ">", "args": [{"property": "vehicle_height"}, {"op": "-", "args": [{"property": "bridge_clearance"}, 1]}]}
```

In both cases, the same CQL2 object representation is built from the expression using AST of R expression evaluation. Then, the object is converted into TEXT or JSON format.

Data types and literal values

A literal value is any part of a CQL2 filter expression used the same as specified in the expression.

The scalar data types are: character string, number, boolean, timestamp, and date.

character string

```
cql2_text("Via dell'Avvento")  
#> 'Via dell'Avvento'  
cql2_json("Via dell'Avvento")  
#> "Via dell'Avvento"
```

number

```
cql2_text(3.1415)  
#> 3.1415  
cql2_json(-100)  
#> -100
```

boolean

```
cql2_text(TRUE)  
#> true  
cql2_json(FALSE)  
#> false
```

timestamp

```
cql2_text(timestamp("1969-07-20T20:17:40Z"))  
#> TIMESTAMP('1969-07-20T20:17:40Z')  
cql2_json(timestamp("1969-07-20T20:17:40Z"))  
#> {"timestamp": "1969-07-20T20:17:40Z"}
```

date

```
cql2_text(date("1969-07-20"))  
#> DATE('1969-07-20')  
cql2_json(date("1969-07-20"))  
#> {"date": "1969-07-20"}
```

Property references

The property of an item can be evaluated in the CQL2 filter expression by its name.

```
cql2_text(windSpeed > 1)
#> windSpeed > 1
cql2_json(windSpeed > 1)
#> {"op": ">", "args": [{"property": "windSpeed"}, 1]}
```

Standard comparison predicates

A comparison predicate evaluates if two scalar expressions satisfy the specified comparison operator.

The standard comparison operators are: =, !=, <, >, <=, >=, and IS NULL.

```
cql2_text(city == "Crato")
#> city = 'Crato'
cql2_json(city == "Jacarei")
#> {"op": "=", "args": [{"property": "city"}, "Jacarei"]}
```

```
cql2_text(avg(windSpeed) < 4)
#> avg(windSpeed) < 4
cql2_json(avg(windSpeed) < 4)
#> {"op": "<", "args": [{"function": {"name": "avg", "args": [{"property": "windSpeed"}]}}, 4]}
```

```
cql2_text(balance - 150.0 > 0)
#> balance - 150 > 0
cql2_json(balance - 150.0 > 0)
#> {"op": ">", "args": [{"op": "-"}, {"property": "balance"}, 150], 0]}
```

```
cql2_text(updated >= date('1970-01-01'))
#> updated >= DATE('1970-01-01')
cql2_json(updated >= date('1970-01-01'))
#> {"op": ">=", "args": [{"property": "updated"}, {"date": "1970-01-01"}]}
```

IS NULL operator

```
cql2_text(!is_null(geometry))
#> geometry IS NOT NULL
cql2_json(!is_null(geometry))
#> {"op": "not", "args": [{"op": "isNull", "args": [{"property": "geometry"}]}]}
```

Advanced comparison operators

A comparison predicate evaluates if two scalar expressions satisfy the specified comparison operator.

Advanced comparison operators are: LIKE, BETWEEN, and IN.

LIKE operator

```
cql2_text(name %like% "Smith%")
#> name LIKE 'Smith%'
cql2_json(name %like% "Smith%")
#> {"op":"like","args":[{"property":"name"},"Smith%"]}
```

BETWEEN operator

```
cql2_text(between(depth, 100.0, 150.0))
#> depth BETWEEN 100 AND 150
cql2_json(between(depth, 100.0, 150.0))
#> {"op":"between","args":[{"property":"depth"},100,150]}
```

IN operator

```
cql2_text(cityName %in% list('Toronto', 'Frankfurt', 'Tokyo', 'New York'))
#> cityName IN ('Toronto','Frankfurt','Tokyo','New York')
cql2_json(cityName %in% list('Toronto', 'Frankfurt', 'Tokyo', 'New York'))
#> {"op":"in","args":[{"property":"cityName"},["Toronto","Frankfurt","Tokyo","New York"]]}
```

```
cql2_text(!category %in% list(1, 2, 3, 4))
#> category NOT IN (1,2,3,4)
cql2_json(!category %in% list(1, 2, 3, 4))
#> {"op":"not","args":[{"op":"in","args":[{"property":"category"},[1,2,3,4]]]}
```

Spatial operators

A spatial predicate evaluates if two spatial expressions satisfy the specified spatial operator.

The supported spatial operators are: S_INTERSECTS, S_EQUALS, S_DISJOINT, S_TOUCHES, S_WITHIN, S_OVERLAPS, S_CROSSES, and S_CONTAINS.

```
poly <- list(type = "Polygon",
             coordinates = list(matrix(c(0,0,0,0,1,1), ncol = 2)))
cql2_text(s_intersects(geometry, {{poly}}))
#> S_INTERSECTS(geometry,POLYGON((0 0,0 1,0 1)))
cql2_json(s_intersects(geometry, {{poly}}))
#> {"op":"s_intersects","args":[{"property":"geometry"},{"type":"Polygon","coordinates": [[[0,0],[0,1],[0,1]]}]}
```

Note: We will use the sfheaders package to represent spatial geometries. The sfheaders package is backward compatible with the sf package. Note: We provide an escape to evaluate user variables using {{ or !!. Both symbols are largely used in the R Data Science community. ### Temporal operators{-}

A temporal predicate evaluates if two temporal expressions satisfy the specified temporal operator.

The supported temporal operators are: T_AFTER, T_BEFORE, T_CONTAINS, T_DISJOINT, T_DURING, T_EQUALS, T_FINISHEDBY, T_FINISHES, T_INTERSECTS, T_MEETS, T_METBY, T_OVERLAPPEDBY, T_OVERLAPS, T_STARTEDBY, and T_STARTS.

```

cql2_text(t_intersects(event_date, interval("1969-07-16T05:32:00Z", "1969-07-24
T16:50:35Z")))
#> T_INTERSECTS(event_date,INTERVAL('1969-07-16T05:32:00Z','1969-07-24T16:50:35Z
'))
cql2_json(t_intersects(event_date, interval("1969-07-16T05:32:00Z", "1969-07-24
T16:50:35Z")))
#> {"op":"t_intersects","args":[{"property":"event_date"}, {"interval
":["1969-07-16T05:32:00Z","1969-07-24T16:50:35Z"]}]}

```

Support for functions in CQL2

Functions allow implementations to extend the language.

Example of a function that returns a geometry value.

```

cql2_text(s_within(road, Buffer(geometry, 10, "m")))
#> S_WITHIN(road,Buffer(geometry,10,'m'))
cql2_json(s_within(road, Buffer(geometry, 10, "m")))
#> {"op":"s_within","args":[{"property":"road"}, {"function":{"name":"Buffer","
args":[{"property":"geometry"},10,"m"]}]}]}

```

Appendix B - rstac CQL2 conformance tests

Here, we list the conformance tests implemented in rstac test suite. The tests run against “Test Dataset for the CQL2 Test Suite” [<https://cql2test.ldproxy.net/ne110m4cql2>] STAC service. rstac implements all conformance tests that have as its purpose to test predicates against the test dataset (except for conformance test #22, which is not present in “Test Dataset for the CQL2 Test Suite”).

Conformance Test	Number of tests
#7	27
#12	14
#16	10
#25	5
#34	12
#38	23
#45	7

References

Finer, Matt, Sidney Novoa, Mikaela J. Weisse, Rachael Petersen, Joseph Mascaro, Tamia Souto, Forest Stearns, and Raúl García Martínez. 2018. “Combating Deforestation: From Satellite to Intervention.” *Science* 360 (6395): 1303–5. <https://doi.org/10.1126/science.aat1203>.

- OGC. 2019. *OGC API - Features - Part 1: Core*. Edited by Clemens Portele, Panagiotis A. Vretanos, and Charles Heazel. Version: 1.0. OGC Implementation Standard. OGC. <http://www.opengis.net/doc/IS/ogcapi-features-1/1.0>.
- . 2022. *Common Query Language (Cql2)*. Edited by Clemens Portele, Panagiotis A. Vretanos, and Charles Heazel. Version: 1.0.0-rc.1. OGC Implementation Standard. OGC. <http://www.opengis.net/doc/IS/cql2/1.0>.
- Simoes, Rolf, Felipe Carvalho de Souza, Matheus Zaglia, Gilberto Ribeiro de Queiroz, Rafael D. C. dos Santos, and Karine Reis Ferreira. 2021. “Rstac: An R Package to Access Spatiotemporal Asset Catalog Satellite Imagery.” In *2021 IEEE International Geoscience and Remote Sensing Symposium IGARSS*, 7674–77. Brussels, Belgium: IEEE. <https://doi.org/10.1109/IGARSS47720.2021.9553518>.
- Wagemann, Julia, Stephan Siemen, Bernhard Seeger, and Jörg Bendix. 2021. “A User Perspective on Future Cloud-Based Services for Big Earth Data.” *International Journal of Digital Earth* 14 (12): 1758–74. <https://doi.org/10.1080/17538947.2021.1982031>.
- Wickham, Hadley. 2019. *Advanced R*. 2nd. ed. CRC press. <https://adv-r.hadley.nz>.